

6 Functions

Figure 6.1 credit: modification of work "IMG_3037", by Jay Roc/Flickr, Public Domain

Chapter Outline

6.1 Defining functions
6.2 Control flow
6.3 Variable scope
6.4 Parameters
6.5 Return values
6.6 Keyword arguments
6.7 Chapter summary

Introduction

Functions are the next step toward creating optimized code as a software developer. If the same block of code is reused repeatedly, a function allows the programmer to write the block of code once, name the block, and use the code as many times as needed by calling the block by name. Functions can read in values and return values to perform tasks, including complex calculations.

Like branching statements discussed in the Decisions chapter, functions allow different paths of execution through a program, and this chapter discusses control flow and the scope of variables in more detail.

6.1 Defining functions

Learning objectives

By the end of this section you should be able to

- Identify function calls in a program.
- Define a parameterless function that outputs strings.
- Describe benefits of using functions.

Calling a function

Throughout the book, functions have been called to perform tasks. Ex: `print()` prints values, and `sqrt()` calculates the square root. A **function** is a named, reusable block of code that performs a task when called.

CHECKPOINT

Example: Simple math program

Example: Simple math program; ch 6, video 1 (https://www.openstax.org/r/simple-math)

CONCEPTS IN PRACTICE

Identifying function calls

1. Which line has a function call?

```
1 input_num = 14
2 offset_num = input_num - 10
3 print(offset_num)
```

   a. line 1
   b. line 2
   c. line 3

2. How many times is print() called?

```
print("Please log in")
username = input("Username:")
password = input("Password:")
print("Login successful")
print("Welcome,", username)
```

   a. 1
   b. 3
   c. 5

3. How many function calls are there?

```
# Use float() to convert input for area calculation
width = float(input("Enter width:"))
height = float(input("Enter height:"))
print("Area is", width*height)
```

   a. 3
   b. 5
   c. 6

Defining a function

A function is defined using the **def** keyword. The first line contains `def` followed by the function name (in snake case), parentheses (with any parameters—discussed later), and a colon. The indented body begins with a documentation string describing the function's task and contains the function statements. A function must

be defined before the function is called.

CHECKPOINT

Example: Welcome message function

Example: Welcome message function; ch 6, video 2 (https://www.openstax.org/r/welcome-message)

CONCEPTS IN PRACTICE

Defining functions

4. What's wrong with the first line of the function definition?

```
def water_plant:
```

   a. A docstring should go after the colon.
   b. Parentheses should go after `water_plant`.
   c. `def` should be `define` before `water_plant`.

5. What is the output?

```
def print_phone_num():
  print("Phone: (", 864, ")", 555, "-", 0199)

print("User info:")
print_phone_num()
```

   a. `Phone: ( 864 ) 555 - 1000`
      `User info:`
      `Phone: ( 864 ) 555 - 1000`
   b. `User info:`
   c. `User info:`
      `Phone: ( 864 ) 555 - 1000`

6. Which statement calls a function named `print_pc_specs()`?
   a. `print_pc_specs`
   b. `print_pc_specs()`
   c. `print_pc_specs():`

7. Which is an appropriate name for a function that calculates a user's taxes?
   a. `calc_tax`
   b. `calculate user tax`
   c. `c_t`

Benefits of functions

A function promotes modularity by putting code statements related to a single task in a separate group. The body of a function can be executed repeatedly with multiple function calls, so a function promotes reusability. Modular, reusable code is easier to modify and is shareable among programmers to avoid reinventing the wheel.

CHECKPOINT

Improving a program with a function

Improving a program with a function; ch 6, video 3 (https://www.openstax.org/r/improving-with-functions)

CONCEPTS IN PRACTICE

Improving programs with functions

Consider the code above.

8. If the points were changed from floats to integers, how many statements would need to be changed in the original and revised programs respectively?
   a. 3, 1
   b. 4, 4
   c. 12, 4
9. How many times can `calc_distance()` be called?
   a. 1
   b. 3
   c. many

TRY IT

Cinema concession stand

Write a function, `concessions()`, that prints the food and drink options at a cinema.

Given:

```
concessions()
```

The output is:

```
Food/Drink Options:
Popcorn: $8-10
Candy: $3-5
Soft drink: $5-7
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-1-defining-functions)

TRY IT

Terms and conditions prompt

Write a function, `terms()`, that asks the user to accept the terms and conditions, reads in `Y/N`, and outputs a response. In the main program, read in the number of users and call `terms()` for each user.

Given inputs `1` and `"Y"`, the output is:

```
Do you accept the terms and conditions?
Y
Thank you for accepting.
```

Given inputs `2`, `"N"`, and `"Y"`, the output is:

```
Do you accept the terms and conditions?
N
Have a good day.
Do you accept the terms and conditions?
Y
Thank you for accepting.
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-1-defining-functions)

6.2 Control flow

Learning objectives

By the end of this section you should be able to

- Identify the control flow of a program.
- Describe how control flow moves between statements and function calls.

Control flow and functions

Control flow is the sequence of program execution. A program's control flow begins at the main program but rarely follows a strict sequence. Ex: Control flow skips over lines when a conditional statement isn't executed.

When execution reaches a function call, control flow moves to where the function is defined and executes the function statements. Then, control flow moves back to where the function was called and continues the sequence.

CHECKPOINT

Calling a brunch menu function

Calling a brunch menu function; ch 6, video 4 (https://www.openstax.org/r/brunch-menu)

CONCEPTS IN PRACTICE

Following the control flow

1. Which line is executed first?

```
1 def park_greet():
2   """Output greeting."""
3   print("Welcome. Open sunrise to sunset.")
4
5 car_count = 1
6 park_greet()
7 if car_count > 50:
8   # Direct to extra parking lot
```

   a. 1
   b. 3
   c. 5

2. Control flow moves to line 9, and `park_greet()` is called. Which line does control flow move to next?

```
 1 def extra_lot():
 2   # Function definition
 3
 4 def park_greet():
 5   """Output greeting."""
 6   print("Welcome. Open sunrise to sunset.")
 7
 8 car_count = 1
 9 park_greet()
10 if car_count > 50:
11   extra_lot()
```

   a. 1
   b. 4
   c. 10

3. Control flow moves to line 12, and `extra_lot()` is called. Which line does control flow move to after line 3 is executed?

```
 1 def extra_lot():
 2   """Output extra parking lot info."""
 3   print("Take the second right to park.")
 4
 5 def park_greet():
 6   """Output greeting."""
 7   print("Welcome. Open sunrise to sunset.")
 8
 9 car_count = 1
10 park_greet()
11 if car_count > 50:
12   extra_lot()
```

   a. 5

b. 8
c. 12

4. What is the output?

```
def park_greet():
  """Output greeting."""
  print("Welcome to the park")
print("Open sunrise to sunset")

park_greet()
```

a.
```
Welcome to the park
```
b.
```
Welcome to the park
  Open sunrise to sunset
```
c.
```
Open sunrise to sunset
  Welcome to the park
```

Functions calling functions

Functions frequently call other functions to keep the modularity of each function performing one task. Ex: A function that calculates an order total may call a function that calculates sales tax. When a function called from another function finishes execution, control flow returns to the calling function.

CHECKPOINT

Example: Book club email messages

Example: Book club email messages; ch 6, video 5 (https://www.openstax.org/r/email-messages)

CONCEPTS IN PRACTICE

Functions calling functions

Consider the book club example above.

5. How many function calls occur during the execution of the program?
   a. 2
   b. 3
   c. 6

6. When line 3 is reached and executed, which line does control flow return to?
   a. 1
   b. 11
   c. 16

TRY IT

Updated terms and conditions prompt

Write an updated function, `terms()`, that asks the user to accept the terms and conditions, reads in `Y/N`, and outputs a response by calling `accepted()` or `rejected()`. `accepted()` prints `"Thank you for accepting the terms."` and `rejected()` prints `"You have rejected the terms. Thank you."`.

Given inputs `2`, `"Y"` and `"N"`, the output is:

```
Do you accept the terms and conditions?
Y
Thank you for accepting the terms.
```

Given a function call to `terms()` and input `"N"`, the output is:

```
Do you accept the terms and conditions?
N
You have rejected the terms. Thank you.
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-2-control-flow)

TRY IT

Laundromat information

Write a program that uses three functions to print information about a laundromat, Liam's Laundry:

- `laundromat_info()`: Prints the name, Liam's Laundry, and hours of operation, 7a - 11p, and calls `washers_open()` and `dryers_open()`
- `washers_open()`: Reads an integer, assigns `washer_count` with the value, and prints `washer_count`
- `dryers_open()`: Reads an integer, assigns `dryer_count` with the value, and prints `dryer_count`

The main program should just call `laundromat_info()`.

Given inputs `50` and `40`, the output is:

```
Liam's Laundry
7a - 11p
 Open washers: 50
 Open dryers: 40
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-2-control-flow)

6.3 Variable scope

Learning objectives

By the end of this section you should be able to

- Identify the scope of a program's variables.
- Discuss the impact of a variable's scope.

Global scope

A variable's **scope** is the part of a program where the variable can be accessed. A variable created outside of a function has **global scope** and can be accessed anywhere in the program. A Python program begins in global scope, and the global scope lasts for the entire program execution.

CHECKPOINT

Global variables in a program with a function

Global variables in a program with a function; ch 6, video 6 (https://www.openstax.org/r/global-variables)

CONCEPTS IN PRACTICE

Global variables

1. Which variables are global?

```
num = float(input())
num_sq = num * num
print(num, "squared is", num_sq)
```

   a. num only
   b. num_sq only
   c. num and num_sq

2. Which variables have global scope?

```
def print_square():
  num_sq = num * num
  print(num, "squared is", num_sq)

num = float(input())
print_square()
```

   a. num only
   b. num_sq only
   c. num and num_sq

3. Which functions can access num?

```
def print_double():
  num_d = num * 2
```

```
  print(num, "doubled is", num_d)

def print_square():
  num_sq = num * num
  print(num, "squared is", num_sq)

num = float(input())
print_double()
print_square()
```

a. `print_double()`
b. `print_square()`
c. `print_double()` and `print_square()`

Local scope

A variable created within a function has **local scope** and only exists within the function. A local variable cannot be accessed outside of the function in which the variable was created. After a function finishes executing, the function's local variables no longer exist.

CHECKPOINT

Global and local variables in a program with a function

Global and local variables in a program with a function; ch 6, video 7 (https://www.openstax.org/r/global-and-local-variables)

CONCEPTS IN PRACTICE

Local variables

4. Which variables are local?

```
def print_time():
  out_str = "Time is " + str(hour) + ":" + str(min)
  print(out_str)

hour = int(input())
min = int(input())
print_time()
```

a. `hour` and `min`
b. `out_str`
c. `hour`, `min`, and `out_str`

5. Which variables are local?

```
def print_greeting():
```

```
  print(out_str)

hour = int(input())
min = int(input())
if hour < 12:
  out_str = "Good morning"
else:
  out_str = "Good day"
print_greeting()
```

a. `hour` and `min`
b. `out_str`
c. none

6. Which functions directly access `out_str`?

```
def print_greeting():
  print("Good day,")
  print_time()

def print_time():
  out_str = "Time is " + str(hour) + ":" + str(min)
  print(out_str)

hour = int(input())
min = int(input())
print_greeting()
```

a. `print_greeting()`
b. `print_time()`
c. `print_greeting()` and `print_time()`

Using local and global variables together

Python allows global and local variables to have the same name, which can lead to unexpected program behavior. A function treats a variable edited within the function as a local variable unless told otherwise. To edit a global variable inside a function, the variable must be declared with the `global` keyword.

CHECKPOINT

Editing global variables in a program with a function

Editing global variables in a program with a function; ch 6, video 8 (https://www.openstax.org/r/editing-global-variables)

CONCEPTS IN PRACTICE

Using both local and global variables

Consider the following variations on the example program with the input 9.

7. What is the output?

```
def update_hour():
  tmp = hour
  if is_dst:
    tmp += 1
  else:
    tmp -= 1

is_dst = True
hour = int(input("Enter hour: "))
update_hour()
print("New hour:", hour)
```

a. New hour: 9
b. New hour: 10
c. Error

8. What is the output?

```
def update_hour():
  new_hour = hour
  if is_dst:
    new_hour += 1
  else:
    new_hour -= 1

is_dst = True
hour = int(input("Enter hour: "))
update_hour()
print("New hour:", new_hour)
```

a. New hour: 9
b. New hour: 10
c. Error

9. What is the output?

```
def update_hour():
  global new_hour
  new_hour = hour
  if is_dst:
    new_hour += 1
  else:
    new_hour -= 1
```

```
is_dst = True
hour = int(input("Enter hour: "))
update_hour()
print("New hour:", new_hour)
```

a. New hour: 9
b. New hour: 10
c. Error

BENEFITS OF LIMITING SCOPE

A programmer might ask, "Why not just make all variables global variables to avoid access errors?" Making every variable global can make a program messy. Ex: A programmer debugging a large program discovers a variable has the wrong value. If the whole program can modify the variable, then the bug could be anywhere in the large program. Limiting a variable's scope to only what's necessary and restricting global variable use make a program easier to debug, maintain, and update.

TRY IT

Battle royale game launch

Write a program that reads in a selected game mode and calls one of two functions to launch the game. If the input is `"br"`, call `battle_royale()`. Otherwise, call `practice()`.

`battle_royale()`:

- Reads in the number of players.
- Computes the number of teammates still needed. A full team is 3 players.
- Calls the function `find_teammates()` with the calculated number.
- Prints `"Match starting . . ."`.

`practice()`:

- Reads in a string representing the desired map.
- Prints `"Launching practice on [desired map]"`.

Note: `find_teammates()` is provided and does not need to be edited.

Given input:

```
br
1
```

The output is:

```
Finding 2 players...
```

```
Match starting...
```

Given input:

```
p
Queen's Canyon
```

The output is:

```
Launching practice on Queen's Canyon
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-3-variable-scope)

6.4 Parameters

Learning objectives

By the end of this section you should be able to

- Identify a function's arguments and parameters.
- Describe how mutability affects how a function can modify arguments.

Arguments and parameters

What if a programmer wants to write a function that prints the contents of a list? Good practice is to pass values directly to a function rather than relying on global variables. A function **argument** is a value passed as input during a function call. A function **parameter** is a variable representing the input in the function definition. Note: The terms "argument" and "parameter" are sometimes used interchangeably in conversation and documentation.

CHECKPOINT

Global and local variables in a program with a function

More global and local variables in a program with a function; ch 6, video 9 (https://www.openstax.org/r/more-global-and-local-variables)

CONCEPTS IN PRACTICE

Arguments and parameters

Consider the following:

```
1 def print_welcome(name):
2   print(f"Welcome {name}!")
3
4 username = int(input("Enter new username: "))
5 print_welcome(username)
```

1. Which is an argument?
   a. `name`
   b. `username`
   c. `name` and `username`
2. Which is a parameter?
   a. `name`
   b. `username`
   c. `name` and `username`
3. What is the scope of `name`?
   a. `print_welcome()` only
   b. whole program
4. What would happen if `username` was changed to `name` on lines 4 and 5?
   a. same output
   b. error/wrong output

Multiple arguments and parameters

Functions can have multiple parameters. Ex: A function uses two parameters, length and width, to compute the square footage of a room. Function calls must use the correct order and number of arguments to avoid undesired behavior and errors (unless using optional or keyword arguments as discussed later).

EXAMPLE 6.1

Using multiple arguments in a function call

```
def print_div(op_1, op_2):
  """ Prints division operation """
  print(f"{op_1}/{op_2} = {op_1/op_2}")

num_1 = 6
num_2 = 3
print_div(num_1, num_2) # Prints "6/3 = 2.0"
print_div(num_2, num_1) # Prints "3/6 = 0.5"
print_div(num_1) # Error: Missing argument: op_2
```

CONCEPTS IN PRACTICE

Multiple arguments and parameters

Consider the following:

```
def calc_distance(x1, y1, x2, y2):
  dist = math.sqrt((x2-x1)**2 + (y2-y1)**2)
```

```
  print(dist)

p1_x =int(input("Enter point 1's x: "))
p1_y =int(input("Enter point 1's y: "))
p2_x =int(input("Enter point 2's x: "))
p2_y =int(input("Enter point 2's y: "))
calc_distance(p1_x, p1_y, p2_x, p2_y)
```

5. Which is an argument?
   a. `p1_x`
   b. `x1`
6. Which is a parameter?
   a. `p1_y`
   b. `y1`
7. What would be the value of x2 for the function call, `calc_distance(2, 4, 3, 6)`?
   a. `2`
   b. `4`
   c. `3`
   d. `6`
   e. `Error`

Modifying arguments and mutability

In Python, a variable is a name that refers to an object stored in memory, aka an object reference, so Python uses a **pass-by-object-reference** system. If an argument is changed in a function, the changes are kept or lost depending on the object's mutability. A **mutable** object can be modified after creation. A function's changes to the object then appear outside the function. An **immutable** object cannot be modified after creation. So a function must make a local copy to modify, and the local copy's changes don't appear outside the function.

Programmers should be cautious of modifying function arguments as these side effects can make programs difficult to debug and maintain.

EXAMPLE 6.2

Converting temperatures

What are the values of `weekend_temps` and `type` after `convert_temps()` finishes?

```
def convert_temps(temps, unit):
  if unit == "F":
    for i in range(len(temps)):
      temps[i] = (temps[i]-32) * 5/9
    unit = "C"
  else:
    for i in range(len(temps)):
      temps[i] = (temps[i]*9/5) + 32
```

```
    unit = "F"

# Weekend temperatures in Fahrenheit.
wknd_temps = [49.0, 51.0, 44.0]
deg_sign = u"\N{DEGREE SIGN}" # Unicode
metric = "F"

# Convert from Fahrenheit to Celsius.
convert_temps(wknd_temps, metric)
for temp in wknd_temps:
  print(f"{temp:.2f}{deg_sign}{metric}", end=" ")
```

The output is `9.44°F 10.56°F 6.67°F`. `type` was changed to `"C"` in the function but didn't keep the change outside of the function. Why is the list argument change kept and not the string argument change? (Hint: A list is mutable. A string is immutable.)

CHECKPOINT

Exploring a faulty function

Exploring a faulty function; ch 6, video 10 (https://www.openstax.org/r/faulty-function)

CONCEPTS IN PRACTICE

Mutability and function arguments

8. In `convert_temps()`, `wknd_temps` and `temps` refer to _____ in memory.
   a. the same object
   b. different objects
9. After `unit` is assigned with `"C"`, `metric` and `unit` refer to _____ in memory.
   a. the same object
   b. different objects
10. `deg_sign` is a string whose value cannot change once created. `deg_sign` is _____.
    a. immutable
    b. mutable
11. On line 16, `unit` _____.
    a. refers to an object with the value `"C"`
    b. does not exist

TRY IT

Printing right triangle area

Write a function, `print_area()`, that takes in the base and height of a right triangle and prints the

triangle's area. The area of a right triangle is $\frac{bh}{2}$, where b is the base and h is the height.

Given input:

```
3
4
```

The output is:

```
Triangle area: 6.0
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-4-parameters)

TRY IT

Curving scores

Write a function, `print_scores()`, that takes in a list of test scores and a number representing how many points to add. For each score, print the original score and the sum of the score and bonus. Make sure not to change the list.

Given function call:

```
print_scores([67, 68, 72, 71, 69], 10)
```

The output is:

```
67 would be updated to 77
68 would be updated to 78
72 would be updated to 82
71 would be updated to 81
69 would be updated to 79
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-4-parameters)

6.5 Return values

Learning objectives

By the end of this section you should be able to

- Identify a function's return value.
- Employ return statements in functions to return values.

Returning from a function

When a function finishes, the function returns and provides a result to the calling code. A **return statement**

finishes the function execution and can specify a value to return to the function's caller. Functions introduced so far have not had a `return` statement, which is the same as returning `None`, representing no value.

CHECKPOINT

Returning a value from a function

Returning a value from a function; ch 6, video 11 (https://www.openstax.org/r/returning-value-from-function)

CONCEPTS IN PRACTICE

Using return statements

1. What is returned by `calc_mpg(miles, gallons)`?

```
def calc_mpg(miles, gallons):
  mpg = miles/gallons
  return mpg
```

   a. `mpg`
   b. `None`
   c. `Error`

2. What is returned by `calc_sqft()`?

```
def calc_sqft(length, width):
  sqft = length * width
  return
```

   a. `sqft`
   b. `None`
   c. `Error`

3. What is the difference between `hw_1()` and `hw_2()`?

```
def hw_1():
  print("Hello world!")
  return

def hw_2():
  print("Hello world!")
```

   a. `hw_1()` returns a string, `hw_2()` does not
   b. `hw_1()` returns `None`, `hw_2()` does not
   c. no difference

Using multiple return statements

Functions that have multiple execution paths may use multiple `return` statements. Ex: A function with an `if-else` statement may have two `return` statements for each branch. `Return` statements always end the

function and return control flow to the calling code.

In the table below, `calc_mpg()` takes in miles driven and gallons of gas used and calculates a car's miles per gallon. `calc_mpg()` checks if gallons is `0` (to avoid division by 0), and if so, returns `-1`, a value often used to indicate a problem.

```
def calc_mpg(miles, gallons):
  if gallons > 0:
    mpg = miles/gallons
    return mpg
  else:
    print("Gallons can't be 0")
    return -1

car_1_mpg = calc_mpg(448, 16)
print("Car 1's mpg is", car_1_mpg)
car_2_mpg = calc_mpg(300, 0)
print("Car 2's mpg is", car_2_mpg)
```

```
Car 1's mpg is 28.0
Gallons can't be 0
Car 2's mpg is -1
```

Table 6.1 Calculating miles-per-gallon and checking for division by zero.

CONCEPTS IN PRACTICE

Multiple return statements

4. What does `yarn_weight(3)` return?

```
def yarn_weight(num):
  if num == 0:
    return "lace"
  elif num == 1:
    return "sock"
  elif num == 2:
    return "sport"
  elif num == 3:
    return "dk"
  elif num == 4:
    return "worsted"
  elif num == 5:
    return "bulky"
  else:
    return "super bulky"
```

   a. `"lace"`
   b. `"dk"`
   c. `"super bulky"`

5. What is the output?

```
def inc_volume(level, max):
  if level < max:
    return level
    level += 1
  else:
    return level

vol1 = inc_volume(9, 10)
print(vol1)
vol2 = inc_volume(10, 10)
print(vol2)
```

a. 9
10
b. 10
10
c. 10
11

Using functions as values

Functions are objects that evaluate to values, so function calls can be used in expressions. A function call can be combined with other function calls, variables, and literals as long as the return value is compatible with the operation.

CHECKPOINT

Using function calls in expressions

Using function calls in expression; ch 6, video 12 (https://www.openstax.org/r/function-calls-in-expressions)

CONCEPTS IN PRACTICE

Using function values

6. What is the updated value of `bill`?

```
def tax(total):
  return .06 * total

def auto_tip(total):
  return .2 * total

bill = 100.0
bill += tax(bill) + auto_tip(bill)
```

a. 26.0
b. 126.0

7. What is the value of `val2`?

```
def sq(num):
  return num * num

def offset(num):
  return num - 2

val = 5
val2 = sq(offset(val))
```

a. 9
b. 23

TRY IT

Estimated days alive

Write a function, `days_alive()`, that takes in an age in years and outputs the estimated days the user has been alive as an integer. Assume each year has 365.24 days. Use `round()`, which takes a number and returns the nearest whole number.

Then write a main program that reads in a user's age and outputs the result of `days_alive()`.

Given input:

```
21
```

The output is:

```
You have been alive about 7670 days.
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-5-return-values)

TRY IT

Averaging lists

Write a function, `avg_list()`, that takes in a list and returns the average of the list values.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-5-return-values)

6.6 Keyword arguments

Learning objectives

By the end of this section you should be able to

- Describe the difference between positional and keyword arguments.
- Create functions that use positional and keyword arguments and default parameter values.

Keyword arguments

So far, functions have been called using **positional arguments**, which are arguments that are assigned to parameters in order. Python also allows **keyword arguments**, which are arguments that use parameter names to assign values rather than order. When mixing positional and keyword arguments, positional arguments must come first in the correct order, before any keyword arguments.

CHECKPOINT

Using keyword arguments

Using keyword arguments; ch 6, video 13 (https://www.openstax.org/r/keyword-arguments)

CONCEPTS IN PRACTICE

Using keyword and positional arguments

Consider the following function:

```
def greeting(msg, name, count):
  i = 0
  for i in range(0, count):
    print(msg, name)
```

1. Which is the positional argument in `greeting(count=1, name="Ash", msg="Hiya")`?
   a. `count=1`
   b. `name="Ash"`
   c. `msg="Hiya"`
   d. None
2. What is the output of `greeting(count=2, name="Ash", msg="Hiya")`?
   a.
   ```
   Ash Hiya
   Ash Hiya
   ```
   b.
   ```
   Hiya Ash
   Hiya Ash
   ```
3. Which is the positional argument in `greeting("Welcome", count=1, name="Anita")`?
   a. `"Welcome"`
   b. `count=1`
   c. `name="Anita"`
4. Which function call would produce an error?

a. `greeting("Morning", "Morgan", count=3)`
b. `greeting(count=1,"Hi", "Bea")`
c. `greeting("Cheers", "Colleagues", 10)`

Default parameter values

Functions can define default parameter values to use if a positional or keyword argument is not provided for the parameter. Ex: `def season(m, d, hemi="N"):` defines a default value of `"N"` for the hemi parameter. Note: Default parameter values are only defined once to be used by the function, so mutable objects (such as lists) should not be used as default values.

The physics example below calculates weight as a force in newtons given mass in kilograms and acceleration in $\frac{m}{s^2}$. Gravity on Earth is 9.8 $\frac{m}{s^2}$, and gravity on Mars is 3.7 $\frac{m}{s^2}$.

CHECKPOINT

Using default parameter values

Using default parameter values; ch 6, video 14 (https://www.openstax.org/r/default-parameter-values)

CONCEPTS IN PRACTICE

Using default parameter values

Consider the following updated version of `greeting()`:

```
def greeting(msg, name="Friend", count=1):
  i = 0
  for i in range(0, count):
    print(msg, name)
```

5. Which parameters have default values?
   a. `msg`
   b. `name` and `count`
   c. all
6. Which function call is correct?
   a. `greeting()`
   b. `greeting(name="Gina")`
   c. `greeting("Greetings")`
7. What is the output of `greeting(count=0, msg="Hello")`?
   a. `Hello Friend`
   b. nothing
   c. `Error`

PEP 8 RECOMMENDATIONS: SPACING

The PEP 8 style guide recommends no spaces around = when indicating keyword arguments and default parameter values.

TRY IT

Stream donations

Write a function, `donate()`, that lets an online viewer send a donation to a streamer. `donate()` has three parameters:

- `amount`: amount to donate, default value: 5
- `name`: donor's name, default value: "Anonymous"
- `msg`: donor's message, default value: ""

Given:

```
donate(10, "gg")
```

The output is:

```
Anonymous donated 10 credits: gg
```

Write function calls that use the default values along with positional and keyword arguments.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/6-6-keyword-arguments)

6.7 Chapter summary

Highlights from this chapter include:

- Functions are named blocks of code that perform tasks when called and make programs more organized and optimized.
- Control flow is the sequence of program execution. Control flow moves between calling code and function code when a function is called.
- Variable scope refers to where a variable can be accessed. Global variables can be accessed anywhere in a program. Local variables are limited in scope, such as to a function.
- Parameters are function inputs defined with the function. Arguments are values passed to the function as input by the calling code. Parameters are assigned with the arguments' values.
- Function calls can use positional arguments to map values to parameters in order.
- Function calls can use keyword arguments to map values using parameter names in any order.
- Functions can define default parameter values to allow for optional arguments in function calls.
- Python uses a pass-by-object-reference system to assign parameters with the object values referenced by the arguments.
- Functions can use `return` statements to return values back to the calling code.

At this point, you should be able to write functions that have any number of parameters and return a value,

and programs that call functions using keyword arguments and optional arguments.

Construct	Description

Function definition	`def function_name():`
	`  """Docstring"""`
	`  # Function body`

Parameter	`def function_name(parameter_1):` `  # Function body`

Argument	`def function_name(parameter_1):`
	`  # Function body`
	
	`function_name(argument_1)`

Return statement	`def function_name():`
	`  # Function body`
	`  return result # Returns the value of result to the caller`

Variables (scope)	`def function_name(parameter_1):`
	`  # Function body`
	`  local_var = parameter_1 * 5`
	`  return local_var`
	
	`global_var = function_name(arg_1)`

Table 6.2 Chapter 6 reference.

Construct	Description

Keyword arguments	`def function_name(parameter_1, parameter_2):`
	`  # Function body`
	
	`function_name(parameter_2 = 5, parameter_1 = 2)`

Default parameter value	`def function_name(parameter_1 = 100):` `  # Function body`

Table 6.2 Chapter 6 reference.